13. I/O: io, Files, Readers and Writers

13. I/O: io, Files, Readers and Writers

Phase: Phase 2 — Core language depth
Estimated time: 1–2 days
Slug: `13-io-readers-writers`

Learning objectives

- Use io.Reader and io.Writer as the core I/O abstractions
- Stream data without loading entire files when possible
- Compose readers (bufio, limit, multi)
- Always close resources with defer carefully

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

io.Reader and io.Writer unify files, sockets, buffers, and compression. Streaming I/O is how Go services stay memory-bounded.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Read/Write return (n, err); process n before assuming err.
- io.Copy moves bytes between Reader and Writer efficiently.
- bufio reduces syscalls for small reads/writes.
- io.EOF ends a stream — often expected, not fatal.
- defer Close() right after a successful Open.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-i/o: && cd /tmp/gw-i/o:
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import (
    "fmt"
    "io"
    "strings"
)

func main() {
    r := strings.NewReader("hello\nworld\n")
    n, err := io.Copy(io.Discard, r)
    fmt.Println(n, err)
}
```

Run it:

```
go run .
```

Expected output:

```
12 <nil>
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Count lines by scanning an io.Reader.
2. Wrap a reader with bufio.Scanner.
3. Use io.LimitReader to cap bytes read.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

The io contracts

```
type Reader interface { Read(p []byte) (n int, err error) }
type Writer interface { Write(p []byte) (n int, err error) }
```

Almost all I/O in Go is streaming through these interfaces — files, sockets, buffers, compression.

```
f, err := os.Open(path)
if err != nil { return err }
defer f.Close()
return io.Copy(dst, f)
```

bufio

Wrap expensive readers/writers with buffering for small reads/writes.

Errors

`io.EOF` signals end of stream — often expected, not always a hard failure. `io.ErrUnexpectedEOF` means truncated data.

Failure modes to watch while coding

- Loading multi-GB files into memory with os.ReadFile without need
- Forgetting defer Close
- Ignoring partial Write/Read return values

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Python file objects ≈ Readers/Writers but Go interfaces compose more uniformly.
- Prefer streaming over `read().split` for large inputs.
- `with open` ≈ Open + defer Close (check Close errors when it matters).

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

Never ignore the n from Read when err != nil — partial data is common.

Practice checklist

Before the exercise, confirm you can:

1. Explain "I/O: io, Files, Readers and Writers" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Loading multi-GB files into memory with os.ReadFile without need
- Forgetting defer Close
- Ignoring partial Write/Read return values

Exercises

Work in `exercises/13-io-readers-writers/`. Read that folder's `README.md` before you code.

What you will build

Implement the reader-counting stub using io.Reader, not []byte whole-file assumptions.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/13-io-readers-writers/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Why is io.Reader more powerful than "give me []byte"?
2. What does io.Copy do?
3. When is io.EOF expected?

Further reading

- https://pkg.go.dev/io
- https://go.dev/blog/laws-of-reflection